Runbook — Deploy Captain Adel (the RAG Cloud Function)

SECTION 06-operations-it TYPE runbook STATUS active OWNER Founder UPDATED 2026-06-16

Captain Adel's chat UI (`chat.html`) is built and live. This runbook connects the **brain** behind it: the `chat` Cloud Function that retrieves GACAR sections and calls Gemini for the answer.

Until this function is deployed, the chat page is honest about it — it shows *"I'm not fully on duty yet, Captain"* and points users to the searchable Library. Once deployed, **no front-end change is needed**: `chat.js` already calls `/api/chat`, which the Hosting rewrite routes to this function.

What you need first

1. **The Blaze (pay-as-you-go) plan.** Cloud Functions do not run on the free Spark plan. Blaze still has a generous free tier — a low-traffic educational chat costs little — but a billing card must be on file. Firebase Console → your project → **Upgrade** → Blaze. Set a **budget alert** (e.g. SAR 100/month) while you're there.
2. **A Gemini API key.** Go to aistudio.google.com → **Get API key**, create one, copy it. (Keep it secret — it is never committed to the repo.)
3. **Firebase CLI**, already installed and signed in from the Phase 1 deploy. If not: `npm install -g firebase-tools && firebase login`.

One-time deploy

From the repository root (`~/Documents/Claude/flygaca/flygaca`):

```
# 1. Install the function's dependencies
cd functions
npm install
cd ..

# 2. Store the Gemini key as a Firebase secret (it prompts you to paste it)
firebase functions:secrets:set GEMINI_API_KEY

# 3. Deploy the function
firebase deploy --only functions
```

The first deploy also enables the required Google Cloud APIs (Cloud Functions, Cloud Build, Artifact Registry, Secret Manager) — approve the prompts. It takes a few minutes.

When it finishes, the CLI prints the function URL. Confirm it is alive with a plain browser GET — it should return a small JSON health object:

```
https://me-central1-<project>.cloudfunctions.net/chat
→ { "status": "ok", "service": "captain-adel", ... }
```

Go live

The function and the front-end are already wired together. Just redeploy hosting so the `/api/chat` rewrite is published:

```
firebase deploy --only hosting
```

Open `chat.html`, ask a GACAR question (e.g. *"What are the VFR weather minima?"*). Captain Adel should answer with section citations and clickable **Sources** that deep-link into the Library reader.

> Deploying hosting + functions together also works: `firebase deploy --only hosting,functions`.

How it works

The function runs a BM25 lexical search over the GACAR corpus, hands the matching section passages to Gemini (`gemini-2.5-flash`) under Captain Adel's system prompt, and returns the answer plus the sources it consulted. No embeddings, no separate vector database — the index builds from a ~15 MB compressed file at cold start (the function is sized at 2 GiB to hold it resident), then every query runs in milliseconds.

Tuning & cost control

Lever	Where	Default
Model	`CAPTAIN_ADEL_MODEL` env var, or `DEFAULT_MODEL` in `rag/agent.js`	`gemini-2.5-flash`
Max concurrent instances	`maxInstances` in `functions/index.js`	`10`
Region	`region` in `index.js` **and** the rewrite in `firebase.json`	`me-central1`
Tool-call rounds per turn	`MAX_TOOL_ROUNDS` in `rag/agent.js`	`5`
Rate limit — per IP / 10 min	`ADEL_RL_IP` env var	`40`
Rate limit — burst per IP / 30 s	`ADEL_RL_BURST` env var	`6`
Rate limit — per browser session / 10 min	`ADEL_RL_SESSION` env var	`24`

Captain Adel runs on `gemini-2.5-flash` — fast, low-cost, and on the Gemini API **free tier**. `gemini-2.5-pro` gives stronger reasoning but is **not** on the free tier (free-tier quota is 0 — every call 429s); to use it, enable the Gemini API **paid tier** on the key's Google Cloud project, then edit `DEFAULT_MODEL` in `rag/agent.js` and `firebase deploy --only functions`.

If you change the region, change it in **both** `index.js` and the `firebase.json` rewrite, or the rewrite will 404.

Updating later

- **Changed the corpus?** Rebuild `functions/rag/_chunks.json.gz`, then `firebase deploy --only functions`.
- **Changed Captain Adel's persona?** Edit `assistant/captain_adel_system_prompt.md` (source of truth) **and** mirror the change into `functions/rag/system-prompt.js`, then redeploy functions.
- **Rotated the API key?** Re-run `firebase functions:secrets:set GEMINI_API_KEY` and redeploy.

Phase 10 — production hardening

Phase 10 makes Captain Adel safe to expose to real, untrusted traffic. Some of it ships in the code and needs nothing from you; the rest needs the console.

Already in the code — ships with the next `firebase deploy --only functions`

- **Rate limiting** (`functions/rag/ratelimit.js`). A sliding-window limiter runs on every turn: per IP, a short burst window per IP, and per browser session. A blocked request gets HTTP 429 and `chat.js` shows a friendly *"ease off a moment"* message. Limits are tunable with the

`ADEL_RL_*` env vars in the table above. Note the honest limit: state is per Cloud Run instance, so it guards against casual abuse and runaway loops but not a distributed attack — App Check below is the next layer.

- **Input guards** (`functions/rag/guards.js`). Control-character stripping, message and history length caps, and a soft prompt-injection detector — a flagged turn is hardened (a security note is appended to the system instruction) rather than blocked, so genuine questions are never refused by mistake. Injection flags appear in the function logs.
- **Eval harness** (`evals/`). Run it after any change to `functions/rag/` or the system prompt:

  ```
  GEMINI_API_KEY=your_key  node evals/run.js
  ```

  It exits non-zero if any case fails. See `evals/README.md`.

Needs the console — your one-time setup

App Check — stops the function being called by anything other than your real site:

1. Firebase Console → **App Check**. Register your Web app with the **reCAPTCHA Enterprise** (or reCAPTCHA v3) provider; copy the site key.
2. In `chat.html` / a small init script, initialise the App Check SDK with that site key so the browser attaches an App Check token to each request.
3. In `functions/index.js`, verify the `X-Firebase-AppCheck` header with `firebase-admin`'s `appCheck().verifyToken()` and reject requests without a valid token. Roll this out in **monitoring mode** first (log failures, do not block) for a few days, then switch to enforced.
4. Keep `invoker: 'public'` — App Check is the application-layer gate; the Cloud Run invoker setting stays public so the browser can reach the function at all.

Budget alerts — so cost can never run away unnoticed:

1. Firebase Console → **Usage and billing** → set a monthly **budget alert** (e.g. SAR 100) with email thresholds at 50 / 90 / 100 %.
2. Google Cloud Console → **Billing → Budgets & alerts** — confirm the same budget covers Cloud Functions, Cloud Run and Artifact Registry.
3. AI Studio / Google Cloud → the Gemini API project — watch the quota page; `gemini-2.5-flash` free-tier limits are the natural ceiling for now.
4. `maxInstances: 10` in `index.js` is the hard cap on concurrent cost — keep it low until traffic justifies more.

CI regression gate (optional, recommended)

Wire `node evals/run.js` into GitHub Actions so a prompt or model change that breaks citations, refusals or injection resistance fails the build. Store the Gemini key as a repository secret. This is the last open Phase 10 item.

Troubleshooting

Symptom	Likely cause
Chat still says *"not fully on duty"*	Function not deployed, or hosting not redeployed after deploy.
"I couldn't reach my engine"	Function deployed but erroring — check `firebase functions:log --only chat`.
"Ease off a moment, Captain"	Rate limit hit (HTTP 429) — expected under heavy use. Raise the `ADEL_RL_*` env vars if it triggers for normal study traffic.
Logs show *"request was not authenticated"*	The 2nd-gen function isn't public. `invoker: 'public'` must be in the `onRequest` options in `index.js` (it is), then redeploy functions. Or grant `allUsers` the **Cloud Run Invoker** role on the `chat` service.
`GEMINI_API_KEY is not configured` in logs	Secret not set, or function not redeployed after setting it.
Deploy fails on region	`me-central1` not enabled for your project — switch both spots to `us-central1` and redeploy.
429 / quota errors from Gemini	Gemini API rate limit — raise the quota in AI Studio, or lower traffic.

Pre-launch reminders

- Captain Adel is an **educational AI** — the on-page disclaimer says so. Keep it.
- The corpus is GACAR text only; live NOTAMs/weather are deliberately out of scope. The system prompt instructs the captain to decline those.
- Confirm the GACAR redistribution position with the Saudi lawyer (P0-1) before a public launch — see `office/lawyer-brief.md`.

Part of the Fly GACA Phase 2 — Captain Adel walkthrough.